

Object-Oriented Programming (CS-1004)

Final Exam

Date: 8/6/2026

Total Time: 3hrs

Course Instructor(s)

Total Marks: 100

Ms. Sumaiyah Zahid, Ms. Sobia Iftikhar, Ms. Abeer Gauher, Mr. Ghulam Qadir, Mr. Ubaidullah, Mr. Talha Shahid, Mr. Aashir Mehboob, Mr. Saif Ur Rehman, Mr. Syed Ahmed Khan

Total Questions: 04

Roll No

Section

Student Signature

Do not write below this line.

Attempt all questions.

CLO 4: Design and assess small and medium scale C++ / C# programs using object-oriented programming principles.

Q1 : Find the output of the codes given. There are no syntax errors. Assume all the libraries and headers are included. **[3*10 =30 Marks, 40 minutes]**

(i) <pre>class A { public: int x; A(int n) : x(n) {} A& operator++() { ++x; return *this; }; int main() { A a(5); ++a; cout<<a.x; }</pre> Solution: 7	(ii) <pre>class A { public: int x; A() : x(10){ x += 5; }; class B : public A { public: int y; B() : y(x + 10){y *= 2; }; }; int main() { B obj; cout << obj.x << " " << obj.y << endl; return 0; }</pre> Solution: 15 50	(vii) <pre>class Device { public: virtual void run() = 0; virtual ~Device() { cout << "~Device "; } }; class Sensor : public Device { public: void run() override { cout << "Sensor "; throw 10; } ~Sensor() { cout << "~Sensor "; } }; template <class T> void execute(T& obj) { try { obj.run(); } catch (int x) { cout << x << " "; } } int main() { Sensor s; execute(s); }</pre> Solution: Sensor 10 ~Sensor ~Device
(iii) <pre>int main() { ofstream out("x.txt"); out << "PROGRAM"; out.close(); ifstream in("x.txt"); cout << in.tellg() << " "; in.seekg(4); cout << in.tellg(); }</pre> Solution: 0 4	(iv) <pre>int main() { string s; fstream file("data.txt", ios::out); file << "ABCDE"; file.seekp(2); file << "X"; file.close(); ifstream in("data.txt"); in >> s; cout << s; }</pre> Solution: ABXDE	
(v) <pre>class A { int x; public: A(int v = 0) : x(v) {} A operator=(const A& obj) { x = obj.x; return *this;} int get() const { return x; } };</pre>	(vi) <pre>template<int N> class Test { public: static const int value = N + Test<N-1>::value; }; template<> class Test<0> { public: static const int value = 1;</pre>	

National University of Computer and Emerging Sciences
Karachi Campus

<pre>int main() { A a(1), b(2), c(3); a = b = c; cout << a.get() << " " << b.get() << " " << c.get(); } Solution: 3 3 3</pre>	<pre>}; int main() { cout << Test<3>::value; } Solution: 7</pre>	
(viii) <pre>class Array { int* data; public: Array(int n) { data = new int[n]; cout << "C "; } Array& operator=(const Array& other) { cout << "="; delete[] data; data = new int[10]; return *this; } ~Array() { cout << "D "; delete[] data; } }; int main() { Array a(5); a = a; return 0; }</pre> Solution: C = D	(ix) <pre>class Base { public: Base() { cout << "B "; } virtual void show() = 0; virtual ~Base() { cout << "~B "; } }; class Derived : public Base { public: Derived() { cout << "D "; throw "Error"; } void show() override { cout << "Show "; } ~Derived() { cout << "~D "; } }; int main() { try { Derived d; } catch (const char* s) { cout << s << " "; } }</pre> Solution: B D ~B Error	(x) <pre>class Base { public: Base() { cout << "Base Ctor\n"; } virtual ~Base() { cout << "Base Dtor\n"; } virtual void func() = 0; }; class Derived : public Base { public: Derived() { cout << "Derived Ctor\n"; } ~Derived() { cout << "Derived Dtor\n"; } void func() override { cout << "Derived::func()\n"; } }; int main() { Base* ptr = new Derived(); ptr->func(); return 0; }</pre> Solution: Base Ctor Derived Ctor Derived::func()

CLO 4: Design and assess small and medium scale C++ / C# programs using object-oriented programming principles.

Q2: (A) Complete the missing codes based on the code snippets and outputs provided. Assume there is no mistake in the code provided and required libraries are included. **[5*3 = 15 Marks , 20 minutes]**

<pre>class DataProcessor { private: string text; public: DataProcessor(string t) { text = t; } string process() { return text + " " + text; } }; string readFromFile(string fileName) { fstream fin(fileName, ios::in); string str; getline(fin, str); return str; }</pre>	<pre>template<typename T> bool isEqual(T a, T b) { return (a == b); } class Test { int val; public: Test(int v) { val = v; } bool operator == (Test& obj) { return this->val == obj.val; } }; int main() { Test t1(50), t2(50), t3(100);</pre>	<pre>class A { protected: int x; public: A(int v) { x = v; } friend int sum(A a1, A a2); }; class B : public A { public: B(int v) : A(v) {} int get() { return x; } }; int sum(A a1, A a2) { return a1.x + a2.x; } int main() { A* p = new B(10); p->get(); A a1(5), a2(15); cout << sum(a1, a2); }</pre>
--	---	---

<pre> void writeToFile(string fileName, string data) { fstream fout(fileName, ios::out); fout << data; } int main() { string files[2] = {"input.txt", "output.txt"}; DataProcessor obj(readFromFile(files[0])) ; writeToFile(files[1], obj.process()); return 0; } </pre>	<pre> cout << isEqual(t1, t2) << ", " << isEqual(t1, t3); return 0; } </pre>	
Input.txt: Hello World Output on File output.txt: Hello World Hello World	Output: 1, 0	Output: 10 20

CLO 1: Discuss knowledge of underlying concepts of object-oriented paradigm like abstraction, encapsulation, polymorphism, inheritance etc.

Q2: (B) Identify OOP principle violations

[2*5 = 10 Marks, 20 minutes]

Study the following C++ program carefully. The program produces the correct output and executes successfully; however, it contains several Object-Oriented Programming (OOP) design and safety violations.

Tasks:

Identify at least **FIVE OOP principle violations**, design flaws, or unsafe programming practices present in the code.

For each issue:

1. Mention the exact line or code segment responsible.
2. Name the violated concept or issue.
3. Briefly explain why it is problematic.
4. Suggest an appropriate correction or safer alternative for each issue.

```

class Device {
protected:
    string id;

public:
    static int activeDevices; // shared mutable
state
    Device(string i = "UNKNOWN") {
        id = i;
        activeDevices++;
    }
    void status() {
        cout << "Device: " << id << endl;
    }
    ~Device() { // ISSUE: non-virtual destructor
        cout << "Device Shutdown\n";
        activeDevices--;
    }
};

int Device::activeDevices = 0;
class Camera {
public:
    void process() {
        cout << "Scanning Ingredients\n";
    }
};

```

```

class Speaker {
public:
    void proces
        cout <<

    };
    class Sensor {
protected:
        int* temp;
public:
        Sensor() {
            temp =

        }
        ~Sensor() {
            delete

        }
    };
    class KitchenRobot :
        public Device,
        public Camera,
        public Speaker,
        public Sensor {
private:
        int meals;
public:
        // constructor issue (shadowing confusion)

```

Output :
Cooking Meal
Scanning Ingredients
Device: UNKNOWN
Meals: 5
Active Devices: 3
Kitchen Ready
Device Shutdown
Device Shutdown
Device Shutdown

National University of Computer and Emerging Sciences

Karachi Campus

```
KitchenRobot(string id, int meals)
: Device() {
    id = id;          // wrong assignment
(shadowing)
    this->meals = meals;
}
void cook() {
    cout << "Cooking Meal\n";
}
void report() {
    cout << "Meals: " << meals << endl;
}
};
int main() {

    // 1. Memory leak + missing virtual
    destructor issue
    Device* d = new KitchenRobot("KR-01", 5);
    // 2. Unsafe C-style downcasting
    KitchenRobot* chef = (KitchenRobot*)d;
    chef->cook();
    // 3. Ambiguous inheritance

    chef->Camera::process();
    // 4. Object slicing
    Device temp = *chef;
    temp.status();
    // 5. Null pointer risk
    KitchenRobot* helper = nullptr;
    if(helper)
        helper->cook();
    // 6. Unsafe cast
    Device* normal = new Device("Simple");
    KitchenRobot* fakeChef =
        (KitchenRobot*)(normal);
    // 7. Constructor logic issue visible
    chef->report();
    // 8. Static shared-state misuse
    Device another;
    cout << "Active Devices: "
        << Device::activeDevices
        << endl;
    cout << "Kitchen Ready\n";
    delete d;
    return 0;
}
```

Code Area	Violation / Issue
~Device()	Missing virtual destructor
(KitchenRobot*)d	Unsafe C-style cast
Device temp = *chef;	Object slicing
helper = nullptr	Null pointer risk
chef->Camera::process()	Ambiguity due to multiple inheritance
id = id;	Constructor/shadowing bug (wrong initialization)
Device() in constructor init list	Partial/incorrect base initialization
new Device("Simple")	Memory leak (normal never deleted)
static int activeDevices	Shared mutable global-like state / static misuse
Device* d = new KitchenRobot(...) without virtual behavior	Weak polymorphic design

CLO 2: Identify real world problems in terms of objects rather than procedure.

Question 03:

[Marks: 25, 50 Minutes]

FAST NUCES, Karachi is upgrading its campus management system to a modern architecture. You are tasked with designing their core class architecture.

Note: "Impact" (or contribution score) represents how much value, performance, or workload a staff member provides to the campus.

Part A: Campus Architecture & Business Rules

FAST operates several types of entities. Design the classes based on the following rules, applying strict encapsulation so that sensitive data is completely hidden from outside access.

1. The General "CampusEntity" Concept

[Marks: 3]

Every entity in the university, regardless of type, has a nuID (string) and loggedHours (double, initially zero). Every entity must be able to calculate its maximum impact. However, the formula to calculate impact depends entirely on the specific type of entity. Include a constructor to set nuID, a calculateImpact() method returning a rule-dependent double, and a logActivity(double hours) method that validates and adds the hours to loggedHours (returning true if successful, false otherwise).

Crucial Rule: There is no such thing as a "generic" entity in the real world. The system must strictly prevent another programmer from creating a direct object of the base entity concept.

2. The "TeachingStaff" Entity**[Marks: 3]**

Include doubles maxCreditLimit and assignedCredits (initially zero), alongside a constructor to initialize local and inherited data. Implement calculateImpact() returning maxCreditLimit * 4.5, and assignCourse(double credits) which adds strictly positive amounts to assignedCredits without exceeding the limit (returning true on success, false otherwise).

3. The "ResearchStaff" Entity**[Marks: 3]**

Include doubles grantCapacity and currentGrants (initially zero), alongside a constructor to initialize local and inherited data. Implement calculateImpact() returning grantCapacity * 12.0, and allocateGrant(double funds) which adds strictly positive amounts to currentGrants without exceeding capacity (returning true on success, false otherwise).

4. The "FacultyMember" Entity**[Marks: 5]**

FAST employs core Faculty Members who act simultaneously as both Teaching and Research staff. Implement calculateImpact() returning the sum of teaching and research impacts. Implement verifyTeachingTeam(parameters...) which validates a team of teaching assistants (returning false if the team size is ≤ 0 or if any individual team member's impact drops below 50.0, returning true otherwise).

Part B: Generic System Tools**[Marks: 3+4=7]**

FAST's software needs generic tools that can operate on any data type.

1. **Comparison Utility:** Write a function findGreaterImpact that takes two objects of the exact same type. It should compare the impacts and return the object that has the higher impact. The function must be capable of accepting any type as long as both arguments are the same type.
2. **Storage Utility:** Design a class DirectoryManager. It must be able to store 5 entities of any one specific type. Include an integer to track the current count of items and a function addMember(item) that takes an object of that type and adds it to the list (the 5 entities) if there is space available.

Part C: System Execution (mandatory)**[Marks: 4]**

Write a main() function that proves your architecture works:

1. Create one TeachingStaff object and one ResearchStaff object.
2. Call your comparison utility from Part B to compare two TeachingStaff objects, and store the result.
3. Create a DirectoryManager capable of holding polymorphic pointers to any type of entity in the system.
4. Create a FacultyMember dynamically and add it to this manager.

Solution:

```
#include "iostream"
using namespace std;

// =====
// PART A: The Diamond Architecture & States
// =====

class CampusEntity{
protected:
    string nuID;
    double loggedHours;

public:

    CampusEntity(string id) : nuID(id), loggedHours(0.0) {}

    virtual double calculateImpact() = 0;

    bool logActivity(double hours){
        if (hours > 0)
        {
            loggedHours += hours;
            return true;
        }
    }
}
```

```
        return false;
    }

    string getNuID(){
        return nuID;
    }
};

class TeachingStaff : virtual public CampusEntity{
protected:
    double maxCreditLimit;
    double assignedCredits;

public:
    TeachingStaff(string id, double credits)
        : CampusEntity(id), maxCreditLimit(credits), assignedCredits(0.0) {}

    double calculateImpact(){
        return maxCreditLimit * 4.5;
    }

    bool assignCourse(double credits)
    {
        if (credits > 0 && (assignedCredits + credits) <= maxCreditLimit)
        {
            assignedCredits += credits;
            return true;
        }
        return false;
    }
};

class ResearchStaff : virtual public CampusEntity
{
protected:
    double grantCapacity;
    double currentGrants;

public:
    ResearchStaff(string id, double grants)
        : CampusEntity(id), grantCapacity(grants), currentGrants(0.0) {}

    double calculateImpact(){
        return grantCapacity * 12.0;
    }

    bool allocateGrant(double funds)
    {
        if (funds > 0 && (currentGrants + funds) <= grantCapacity)
        {
            currentGrants += funds;
            return true;
        }
        return false;
    }
};

class FacultyMember : public TeachingStaff, public ResearchStaff
```

```

{
public:
    FacultyMember(string id, double credits, double grants)
        : CampusEntity(id), TeachingStaff(id, credits), ResearchStaff(id, grants) {}

    double calculateImpact(){
        return TeachingStaff::calculateImpact() + ResearchStaff::calculateImpact();
    }

    bool verifyTeachingTeam(TeachingStaff team[], int size){
        if (size <= 0){
            return false;
        }

        for (int i = 0; i < size; i++){
            if (team[i].calculateImpact() < 50.0)
            {
                return false;
            }
        }
        return true;
    }
};

// =====
// PART B: Generic Programming (Templates)
// =====

// Template Function
template <typename T>
T findGreaterImpact(T a, T b){
    if (a.calculateImpact() > b.calculateImpact())
    {
        return a;
    }
    return b;
}

// Template Class
template <typename T>
class DirectoryManager{
private:
    T members[5];
    int count;

public:
    DirectoryManager() : count(0) {}

    void addMember(T item){
        if (count < 5){
            members[count] = item;
            count++;
        }
        else{
            cout << "Directory Manager is full!" << endl;
        }
    }
}

```

```
int getCount(){
    return count;
}
};

// =====
// PART C: System Execution (main)
// =====

int main(){
    cout << "--- FAST NUCES System Booting ---" << endl;

    // 1. Instantiate objects and test state validations
    TeachingStaff ts1("NU-100", 80.0);
    ResearchStaff rs1("NU-200", 40.0);

    cout << "TS1 Impact: " << ts1.calculateImpact() << endl;
    cout << "Assigning TS1 50 credits: " << (ts1.assignCourse(50.0) ? "Success" :
"Failed") << endl;
    cout << "Assigning TS1 100 credits (Over limit test): " << (ts1.assignCourse(100.0)
? "Success" : "Failed") << endl;

    // 2. Test Template Function
    TeachingStaff ts2("NU-101", 90.0);
    TeachingStaff betterTS = findGreaterImpact<TeachingStaff>(ts1, ts2);

    cout << "Staff with greater impact has ID: " << betterTS.getNuID() << endl;

    // 3. Test Faculty Member and Array of Objects
    FacultyMember fm("FAC-300", 30.0, 20.0);
    cout << "Faculty Impact: " << fm.calculateImpact() << endl;

    TeachingStaff team[3] = {
        TeachingStaff("TA-1", 60.0), // Impact: 270
        TeachingStaff("TA-2", 80.0), // Impact: 360
        TeachingStaff("TA-3", 10.0) // Impact: 45 (Will cause verify to fail)
    };

    cout << "Team Validity Check: "
        << (fm.verifyTeachingTeam(team, 3) ? "Valid to proceed" : "Invalid -
insufficient impact") << endl;

    // 4. Test Template Class (Polymorphism using pointers)
    DirectoryManager<CampusEntity *> myDirectory;

    CampusEntity *ptr1 = &ts1;
    CampusEntity *ptr2 = &fm;

    myDirectory.addMember(ptr1);
    myDirectory.addMember(ptr2);

    cout << "Entities registered in DirectoryManager: " << myDirectory.getCount() <<
endl;
}
```

CLO 5: Synthesize programs using Generic Programming and exception handling.

Question 04:

[Marks: 20, 50 Minutes]

A smart grid records hourly energy consumption from various households into a text file (energy_log.txt). Each line in the file represents a single log entry and must follow a strict comma-separated format:

[Timestamp],[Household_ID],[Energy_Consumed_kWh]

Example of a valid file:

2026-05-18 08:00,HH-102,4.5
2026-05-18 09:00,HH-103,12.8

However, due to occasional wireless sensor interference, the log file frequently suffers from data corruption, missing values, or impossible telemetry readings. Ensure that file handles for both the input logs and the corruption report are guaranteed to close safely under all circumstances whether the script runs perfectly, encounters data errors, or faces sudden system termination (like a missing file).

Part A: Custom Exceptions

[2+2+2 = 6 Marks]

Design and implement a custom exception hierarchy. You must create a base user defined exception class and at least two specific sub classes to handle different categories of failure:

1. The base exception class should extend from the standard “Exception” parent class. It contains **line number** and **message** as protected attributes. Add constructors and override the what() function to display the error message.
2. One sub class to handle data formatting or syntax anomalies (e.g., missing fields, wrong delimiters). Override the what() function to display the error message.
3. One sub class to handle business logic anomalies (e.g., negative energy values, values exceeding physically possible limits like 500 kWh/hour). Override the what() function to display the error message.

Part B: Filing**[2.5 + 2.5 + 2.5 + 2.5 = 10 Marks]**

Create a **LogProcessor** class. This class should encapsulate the data for a single processed entry and contain private attributes with appropriate public getter methods.

1. All attributes (timestamp, householdID, and energyConsumed) are private. Implement the constructor. It should also include validation logic and use the exception classes made in part a.
2. writeRecordToFile(string filepath, LogProcessor& log): Writes the log processor records using this function.
3. updateRecordInFile(string filepath, string targetID, double newEnergyValue): Searches for a record by its unique Household ID and updates its energy usage.
4. deleteRecordFromFile(string filepath, string targetID): Finds a specific target ID and removes it from the file

Part C: System Execution (mandatory)**[4 Marks]**

1. Implement a file processing loop inside a global method named **process_logs(input_filepath, corruption_filepath)**.
2. The method must open and read energy_log.txt line by line. If a single line is corrupted, the system must not stop processing the remaining lines.
3. If an exception is caught, your script should handle it by logging the exact line number, the raw corrupted text, and the specific reason for failure into a separate file (corruption_report.txt).
4. Successfully parsed data should be printed to the console.

Solution:

```
// =====
// PART A: Custom Exception Hierarchy
// =====
```

```
class EnergyLogException : public exception {
protected:
    int _lineNumber;
    string _message;

public:
    EnergyLogException(int lineNumber, string& message)
        : _lineNumber(lineNumber), _message(message) {}
```

```
void setLineNumber(int lineNumber) { _lineNumber = lineNumber; }
void setMessage(string& message) { _message = message; }

int getLineNumber() const { return _lineNumber; }
string getMessage() const { return _message; }

// Override what()
virtual const char* what() {
    string fullMsg;
    fullMsg = "[EnergyLogException] Line " + to_string(_lineNumber) + ": " + _message;
    return fullMsg.c_str();
}

virtual ~EnergyLogException() = default;
};

// — Sub-class 1: Format/Syntax Anomalies —
class FormatException : public EnergyLogException {
public:
    FormatException(int lineNumber, string& message)
        : EnergyLogException(lineNumber, message) {}

    const char* what() const override {
        string fullMsg;
        fullMsg = "[FormatException] Line " + to_string(_lineNumber) + " - Syntax/Format Error: " + _message;
        return fullMsg.c_str();
    }
};

// — Sub-class 2: Business Logic Anomalies —
class ValueException : public EnergyLogException {
public:
    ValueException(int lineNumber, string& message)
        : EnergyLogException(lineNumber, message) {}

    const char* what() const override {
        static std::string fullMsg;
        fullMsg = "[ValueException] Line " + to_string(_lineNumber) + " - Business Logic Error: " + _message;
        return fullMsg.c_str();
    }
};
```

```
// =====  
// PART B: LogProcessor Class  
// =====  
  
class LogProcessor {  
private:  
    string _timestamp;  
    string _householdID;  
    double _energyConsumed;  
  
public:  
    // — Constructor with validation —  
    LogProcessor(string& timestamp, string& householdID, string& energyStr, int lineNumber)  
    {  
        // Validate timestamp not empty  
        if (timestamp.empty())  
            throw FormatException(lineNumber, "Timestamp field is empty.");  
  
        // Validate householdID not empty  
        if (householdID.empty())  
            throw FormatException(lineNumber,  
                "Household ID field is empty.");  
  
        // Business logic: no negative energy  
        if (energyVal < 0.0)  
            throw ValueException(lineNumber, "Energy value " + energyStr + " kWh is negative, which is  
physically impossible.");  
  
        // Business logic: max 500 kWh/hour  
        if (energyVal > 500.0)  
            throw ValueException(lineNumber, "Energy value " + energyStr +  
                " kWh exceeds maximum limit of 500 kWh/hour.");  
  
        _timestamp = timestamp;  
        _householdID = householdID;  
        _energyConsumed = energyVal;  
    }  
  
    string getTimestamp() const { return _timestamp; }  
    string getHouseholdID() const { return _householdID; }  
    double getEnergyConsumed()const { return _energyConsumed; }
```

```
void display() const {
    std::cout << " Timestamp : " << _timestamp << "\n"
        << " HouseholdID : " << _householdID << "\n"
        << " Energy(kWh) : " << _energyConsumed << "\n";
}
```

```
// — Write record to file —
static void writeRecordToFile(const std::string& filepath, LogProcessor& log)
{
    ofstream outFile(filepath, ios::app);
    if (!outFile.is_open())
        throw runtime_error("Cannot open file for writing: " + filepath);
```

```
    outFile << log.getTimestamp() << ","
        << log.getHouseholdID() << ","
        << log.getEnergyConsumed() << "\n";
```

```
    outFile.close();
    cout << "[WriteRecord] Record written for HouseholdID: " << log.getHouseholdID() << "\n";
}
```

```
// — Update record in file —
static void updateRecordInFile(string& filepath, string& targetID, double newEnergyValue)
{
```

```
    // Validate new value
    if (newEnergyValue < 0.0 || newEnergyValue > 500.0)
        throw ValueException(0, "New energy value " + to_string(newEnergyValue) + " is out of valid range [0, 500].");
```

```
    ifstream inFile(filepath);
    if (!inFile.is_open())
        throw runtime_error("Cannot open file for reading: " + filepath);
```

```
    vector<string> lines;
    string line;
    bool found = false;
```

```
    while (getline(inFile, line)) {
        if (line.empty()) { lines.push_back(line); continue; }
```

```
        stringstream ss(line);
        string ts, hid, eng;
        getline(ss, ts, ',');
        getline(ss, hid, ',');
        getline(ss, eng, ',');
```

```
        if (hid == targetID) {
```

```
// Replace energy value
line = ts + "," + hid + "," + to_string(newEnergyValue);
found = true;
}
lines.push_back(line);
}
inFile.close();

if (!found) {
    cout << "[UpdateRecord] HouseholdID '" << targetID << "' not found.\n";
    return;
}

// Write all lines back
ofstream outFile(filepath, ios::trunc);
if (!outFile.is_open())
    throw std::runtime_error("Cannot open file for writing: " + filepath);

for (const auto& l : lines)
    outFile << l << "\n";

outFile.close();
cout << "[UpdateRecord] Updated HouseholdID '" << targetID << "' to '" << newEnergyValue << "
kWh.\n";
}

// — Delete record from file —
static void deleteRecordFromFile(string& filepath, string& targetID)
{
    ifstream inFile(filepath);
    if (!inFile.is_open())
        throw runtime_error("Cannot open file for reading: " + filepath);

    vector<string> lines;
    string line;
    bool found = false;

    while (getline(inFile, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string ts, hid, eng;
        getline(ss, ts, ',');
        getline(ss, hid, ',');
        getline(ss, eng, ',');

        if (hid == targetID) {
```

```
        found = true;
        continue;
    }
    lines.push_back(line);
}
inFile.close();

if (!found) {
    cout << "[DeleteRecord] HouseholdID '" << targetID << "' not found.\n";
    return;
}

ofstream outFile(filepath, ios::trunc);
if (!outFile.is_open())
    throw std::runtime_error("Cannot open file for writing: " + filepath);

for (const auto& l : lines)
    outFile << l << "\n";

outFile.close();
std::cout << "[DeleteRecord] Deleted record for HouseholdID '"
    << targetID << " ".\n";
}
};
```

```
// =====
// PART C: process logs()
// =====
```

```
void process_logs(string& input_filepath, string& corruption_filepath)
{
    ifstream* inFile = nullptr;
    ofstream* reportFile = nullptr;

    try {
        inFile = new ifstream(input_filepath);
        if (!inFile->is_open()) {
            throw runtime_error("FATAL: Cannot open input file: " + input_filepath);
        }

        // Open corruption report file
        reportFile = new ofstream(corruption_filepath, ios::app);
        if (!reportFile->is_open()) {
```

```

    throw runtime_error( "FATAL: Cannot open corruption report: " + corruption_filepath);
}

string line;
int lineNumber = 0;

cout << "\n==== Processing Energy Logs =====\n";

while (getline(*inFile, line)) {
    lineNumber++;

    if (line.empty()) continue;

    try {
        // — Parse CSV fields —
        stringstream ss(line);
        string timestamp, householdID, energyStr;

        bool gotTS = (bool)getline(ss, timestamp, ',');
        bool gotHID = (bool)getline(ss, householdID, ',');
        bool gotEng = (bool)getline(ss, energyStr, ',');

        // Check we got exactly 3 fields
        if (!gotTS || !gotHID || !gotEng) {
            throw FormatException(lineNumber, "Expected 3 comma-separated fields, got fewer.");
        }

        // Check no extra fields
        string extra;
        if (getline(ss, extra, ',') && !extra.empty()) {
            throw FormatException(lineNumber, "Too many fields detected in line.");
        }

        // — Construct LogProcessor (validates inside) —
        LogProcessor record(timestamp, householdID, energyStr, lineNumber);
        cout << "\n[Line " << lineNumber << "] Valid Record:\n";
        record.display();

    }

    // Catch format errors for this line
    catch (const FormatException& fe) {
        cerr << fe.what() << "\n";
        *reportFile << "[Line " << lineNumber << "] "<< "RAW: \"" << line << "\" |<< "REASON: " <<
fe.what() << "\n";
    }

    // Catch value/business logic errors for this line
    catch (const ValueException& ve) {

```

```
        cerr << ve.what() << "\n";
        *reportFile << "[Line " << lineNumber << "] << "RAW: \"" << line << "\" | " << "REASON: " <<
ve.what() << "\n";
    }
    // Catch any other per-line error
    catch (const EnergyLogException& ele) {
        cerr << ele.what() << "\n";
        *reportFile << "[Line " << lineNumber << "] "
        << "RAW: \"" << line << "\" | "
        << "REASON: " << ele.what() << "\n";
    }
}
```

```
    cout << "\n==== Processing Complete ==== \n";
    cout << "Total lines read: " << lineNumber << "\n";
    cout << "Corruption report saved to: " << corruption_filepath << "\n";
}
// Fatal errors (file not found, etc.)
catch (const runtime_error& rte) {
    cerr << "\n[FATAL ERROR] " << rte.what() << "\n";
}
catch (...) {
    cerr << "\n[UNKNOWN FATAL ERROR] << "Unexpected error during processing.\n";
}
```